BEM Solver для электромагнитного рассеяния

Подробное руководство: физика, формулы, реализация

Содержание

1 Введение **2**
- 1.1 Состав проекта . . . 2
- 1.2 Зависимости . . . 2

2 Физика задачи **2**
- 2.1 Постановка: рассеяние плоской волны на частице . . . 2
- 2.2 Уравнения Максвелла и интегральные уравнения . . . 3
- 2.3 PMCHWT-формулировка . . . 3
- 2.4 Случай PEC (EFIE) . . . 3

3 Дискретизация: RWG базисные функции **3**
- 3.1 Определение RWG функций . . . 3
- 3.2 Дивергенция RWG функций . . . 4
- 3.3 Построение в коде . . . 4
- 3.4 Масштаб задачи . . . 5

4 Генерация меша: икосфера **5**

5 Квадратуры на треугольнике **5**

6 Сборка матрицы: операторы L и K **6**
- 6.1 Дискретный оператор L . . . 6
- 6.2 Дискретный оператор K . . . 6

7 Обработка сингулярностей **6**
- 7.1 Проблема . . . 6
- 7.2 Метод: извлечение сингулярности (Graglia 1993) . . . 6
- 7.3 Аналитический интеграл (Graglia) . . . 7
- 7.4 Какие интегралы обрабатываются аналитически . . . 7
- 7.5 Оператор K: сингулярная часть . . . 7
- 7.6 Градиент потенциала (конечные разности) . . . 8

8 Сборка PMCHWT системы **8**

9 Правая часть: падающая волна **8**

10 Решение системы и дальнее поле **8**
10.1 Решение линейной системы . . . 8
10.2 Вычисление дальнего поля . . . 9

11 Сечения рассеяния **9**
11.1 Дифференциальное сечение . . . 9
11.2 Полное сечение рассеяния . . . 9
11.3 Эффективность рассеяния . . . 9
11.4 Оптическая теорема . . . 9

12 Теория Ми (валидация) **10**
12.1 Коэффициенты Ми . . . 10
12.2 Эффективности . . . 10
12.3 PEC-сфера . . . 10

13 Сравнение методов **10**
13.1 BEM (собственная реализация и bempp-cl) . . . 10
13.2 DDA / ADDA . . . 11
13.3 Ключевые выводы из сравнений . . . 11

14 Солверы и предобуславливание **11**
14.1 LU-разложение (прямой метод) . . . 11
14.2 GMRES . . . 11
14.3 FMM-ускорение . . . 11

15 Как запускать **11**
15.1 Базовый тест (собственный BEM) . . . 11
15.2 Сравнение с bempp-cl и ADDA . . . 12
15.3 Типичный вывод . . . 12

16 Ограничения и возможные улучшения **12**

17 Список литературы **13**

1 Введение

Данный проект реализует **метод граничных элементов** (BEM, Boundary Element Method) для решения задачи рассеяния электромагнитных волн на диэлектрических и идеально проводящих (PEC) телах. Реализация выполнена на чистом Python (NumPy/SciPy).

1.1 Состав проекта

- `bem_core.py` — ядро солвера: меш, базисные функции, сборка матрицы, решение
- `test_sphere.py` — тест на диэлектрической сфере, сравнение с теорией Ми
- `test_pec.py` — тест на PEC-сфере (EFIE-формулировка)
- `compare_final.py` — сравнение BEM vs DDA (ADDA) vs Mie
- `fast_bempp.py` — оптимизация GMRES с библиотекой bempp-cl
- `compare_fmm.py`, `fmm_test*.py` — тесты FMM-ускорения

1.2 Зависимости

- `numpy`, `scipy` — основные вычисления
- `bempp-cl` — только для файлов сравнения (не нужен для `bem_core.py`)
- ADDA — внешний DDA-солвер для сравнения

2 Физика задачи

2.1 Постановка: рассеяние плоской волны на частице

Рассматривается однородная диэлектрическая частица с показателем преломления $m = n_1/n_0$ (относительным), помещённая в однородную среду. На частицу падает плоская электромагнитная волна:

$$\mathbf{E}^{\text{inc}}(\mathbf{r}) = \mathbf{E}_0\, e^{ik_{\text{ext}}\hat{\mathbf{k}}\cdot\mathbf{r}}, \qquad \mathbf{H}^{\text{inc}}(\mathbf{r}) = \frac{\hat{\mathbf{k}} \times \mathbf{E}_0}{\eta_{\text{ext}}}\, e^{ik_{\text{ext}}\hat{\mathbf{k}}\cdot\mathbf{r}} \tag{1}$$

где $k_{\text{ext}} = \omega\sqrt{\mu_0\varepsilon_{\text{ext}}}$ — волновое число в среде, $\eta_{\text{ext}} = \sqrt{\mu_0/\varepsilon_{\text{ext}}}$ — импеданс среды.

Внутри частицы:

$$k_{\text{int}} = m \cdot k_{\text{ext}}, \qquad \eta_{\text{int}} = \eta_{\text{ext}}/m \qquad (\text{при } \mu_r = 1) \tag{2}$$

2.2 Уравнения Максвелла и интегральные уравнения

Из уравнений Максвелла в каждой области (внешней и внутренней) и теоремы эквивалентности Гюйгенса следует, что поля однозначно определяются **тангенциальными компонентами** на границе S:

$$\mathbf{J} = \hat{\mathbf{n}} \times \mathbf{H}, \qquad \mathbf{M} = -\hat{\mathbf{n}} \times \mathbf{E} \tag{3}$$

где $\hat{\mathbf{n}}$ — внешняя нормаль к поверхности, $\mathbf{J}$ — эквивалентный электрический ток, $\mathbf{M}$ — эквивалентный магнитный ток.

Поля выражаются через поверхностные токи с помощью операторов:

$$\mathcal{L}_k[\mathbf{X}](\mathbf{r}) = ik \int_S G_k(\mathbf{r}, \mathbf{r}')\mathbf{X}(\mathbf{r}')\, dS' + \frac{i}{k}\nabla \int_S G_k(\mathbf{r}, \mathbf{r}')\nabla' \cdot \mathbf{X}(\mathbf{r}')\, dS' \tag{4}$$

$$\mathcal{K}_k[\mathbf{X}](\mathbf{r}) = \int_S \nabla G_k(\mathbf{r}, \mathbf{r}') \times \mathbf{X}(\mathbf{r}')\, dS' \tag{5}$$

где $G_k(\mathbf{r}, \mathbf{r}') = \frac{e^{ik|\mathbf{r}-\mathbf{r}'|}}{4\pi|\mathbf{r}-\mathbf{r}'|}$ — скалярная функция Грина свободного пространства.

Оператор $\mathcal{L}$ называется **EFIE-оператором** (Electric Field Integral Equation), а $\mathcal{K}$ — **MFIE-оператором** (Magnetic Field Integral Equation).

2.3 PMCHWT-формулировка

Формулировка Погорелова–Мюллера–Чу–Харрингтона–Ву–Цая (PMCHWT, Poggio–Miller–Chang–Harrington–Wu–Tsai) получается из требования непрерывности тангенциальных полей на границе. Складывая вклады от внешней и внутренней областей:

$$\boxed{\begin{pmatrix} \eta_1\mathcal{L}_1 + \eta_2\mathcal{L}_2 & \mathcal{K}_1 + \mathcal{K}_2 \\ -(\mathcal{K}_1 + \mathcal{K}_2) & \dfrac{\mathcal{L}_1}{\eta_1} + \dfrac{\mathcal{L}_2}{\eta_2} \end{pmatrix} \begin{pmatrix} \mathbf{J} \\ \mathbf{M} \end{pmatrix} = \begin{pmatrix} \hat{\mathbf{n}} \times \mathbf{E}^{\text{inc}} \\ \hat{\mathbf{n}} \times \mathbf{H}^{\text{inc}} \end{pmatrix}} \tag{6}$$

Здесь индекс 1 = ext (внешняя среда), индекс 2 = int (частица). Эта система $2N \times 2N$ (где N — число базисных функций) решается относительно коэффициентов разложения токов $\mathbf{J}$ и $\mathbf{M}$.

2.4 Случай PEC (EFIE)

Для идеально проводящей сферы $\mathbf{M} = 0$ (поле внутри PEC равно нулю), и система упрощается до **EFIE**:

$$\eta\, \mathcal{L}_{\text{ext}}[\mathbf{J}] = \hat{\mathbf{n}} \times \mathbf{E}^{\text{inc}} \tag{7}$$

Это система $N \times N$ только для электрического тока $\mathbf{J}$.

3 Дискретизация: RWG базисные функции

3.1 Определение RWG функций

Базисные функции Рао–Уилтона–Глиссона (RWG, Rao–Wilton–Glisson, 1982) определены на парах смежных треугольников, разделяющих общее ребро. Для n-й базисной функции, ассоциированной с ребром длины l_n, соединяющим треугольники T_n^+ и T_n^-:

$$\mathbf{f}_n(\mathbf{r}) = \begin{cases} \dfrac{l_n}{2A_n^+}(\mathbf{r} - \mathbf{r}_n^+), & \mathbf{r} \in T_n^+ \\ -\dfrac{l_n}{2A_n^-}(\mathbf{r} - \mathbf{r}_n^-), & \mathbf{r} \in T_n^- \\ 0, & \text{иначе} \end{cases} \tag{8}$$

где:

- l_n — длина общего ребра
- $A_n^\pm$ — площади треугольников $T_n^\pm$
- $\mathbf{r}_n^+$ — вершина T_n^+, противоположная общему ребру ("свободная вершина")
- $\mathbf{r}_n^-$ — аналогичная вершина T_n^-

Знаки выбраны так, что ток "вытекает" из T_n^+ через ребро и "втекает" в T_n^-, обеспечивая непрерывность нормальной компоненты на ребре.

3.2 Дивергенция RWG функций

Дивергенция — кусочно-постоянная:

$$\nabla \cdot \mathbf{f}_n = \begin{cases} +\dfrac{l_n}{A_n^+}, & \mathbf{r} \in T_n^+ \\ -\dfrac{l_n}{A_n^-}, & \mathbf{r} \in T_n^- \end{cases} \tag{9}$$

3.3 Построение в коде

Функция `build_rwg(verts, tris)` обходит все рёбра меша. Для каждого **внутреннего** ребра (принадлежащего двум треугольникам) создаётся одна RWG-функция. Граничные рёбра (один треугольник) пропускаются — для замкнутых поверхностей их нет.

Результат — словарь:

- `N` — число RWG-функций (= число внутренних рёбер)
- `tri_p`, `tri_m` — индексы треугольников T^+ и T^-
- `free_p`, `free_m` — координаты свободных вершин
- `length` — длины рёбер l_n
- `area_p`, `area_m` — площади треугольников

3.4 Масштаб задачи

Для икосферы с r уровнями измельчения:

Refinements	Triangles	Vertices	RWG functions
1	80	42	120
2	320	162	480
3	1280	642	1920
4	5120	2562	7680

Размер системы PMCHWT: $2N \times 2N$, т.е. для refine=3 это 3840×3840.

4 Генерация меша: икосфера

Функция `icosphere(radius, refinements)` строит триангуляцию сферы:

1. Начальный икосаэдр: 12 вершин, 20 треугольников.
2. Каждое измельчение: каждый треугольник делится на 4 (вставкой средних точек рёбер).
3. Новые вершины проецируются на сферу (нормируются к единице).
4. Финальное масштабирование на `radius`.

Число треугольников после r измельчений: 20×4^r.
Координаты вершин икосаэдра используют золотое сечение $\varphi = (1 + \sqrt{5})/2$:

$$(\pm 1, \pm\varphi, 0), \quad (0, \pm 1, \pm\varphi), \quad (\pm\varphi, 0, \pm 1) \tag{10}$$

(все перестановки и комбинации знаков, дающие 12 вершин).

5 Квадратуры на треугольнике

Интегралы по треугольникам вычисляются численно с помощью квадратурных формул на стандартном треугольнике $\{(\xi_1, \xi_2) : \xi_1 \geq 0, \xi_2 \geq 0, \xi_1 + \xi_2 \leq 1\}$.

Точка внутри треугольника с вершинами $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2$:

$$\mathbf{r}(\xi_1, \xi_2) = (1 - \xi_1 - \xi_2)\mathbf{v}_0 + \xi_1\mathbf{v}_1 + \xi_2\mathbf{v}_2 \tag{11}$$

Реализованы две схемы:

- **4-точечная** (порядок 3): центроидальная + 3 точки. Веса: $-27/48$, $25/48$.
- **7-точечная** (порядок 5): 1 центральная + 2 орбиты по 3 точки. Используется по умолчанию.

Якобиан преобразования: $dS = 2A\, d\xi_1\, d\xi_2$, где A — площадь треугольника.

6 Сборка матрицы: операторы L и K

6.1 Дискретный оператор L

Тестирование оператора $\mathcal{L}$ на RWG-функциях даёт матричный элемент:

$$L_{mn} = ik \underbrace{\int_S \int_S \mathbf{f}_m(\mathbf{r}) \cdot \mathbf{f}_n(\mathbf{r}')\, G_k(R)\, dS\, dS'}_{\text{векторная часть}} - \frac{i}{k} \underbrace{\int_S \int_S (\nabla \cdot \mathbf{f}_m)(\nabla' \cdot \mathbf{f}_n)\, G_k(R)\, dS\, dS'}_{\text{скалярная (div-div) часть}} \tag{12}$$

где $R = |\mathbf{r} - \mathbf{r}'|$.

Каждый элемент включает интегрирование по 4 парам полу-треугольников: (T_m^+, T_n^+), (T_m^+, T_n^-), (T_m^-, T_n^+), (T_m^-, T_n^-).

6.2 Дискретный оператор K

$$K_{mn} = \int_S \int_S \mathbf{f}_m(\mathbf{r}) \cdot \left[\nabla G_k(R) \times \mathbf{f}_n(\mathbf{r}')\right] dS\, dS' \tag{13}$$

Градиент функции Грина:

$$\nabla G_k(R) = \frac{e^{ikR}}{4\pi R} \left(ik - \frac{1}{R}\right) \hat{\mathbf{R}} \tag{14}$$

где $\hat{\mathbf{R}} = (\mathbf{r} - \mathbf{r}')/R$.

7 Обработка сингулярностей

7.1 Проблема

При совпадении треугольников наблюдения и источника ($T_m = T_n$) подынтегральные выражения содержат сингулярности:

- $G_k \sim 1/(4\pi R)$ при $R \to 0$ — **интегрируемая** особенность (слабая)
- $\nabla G_k \sim 1/(4\pi R^2)$ при $R \to 0$ — **сильная** особенность

7.2 Метод: извлечение сингулярности (Graglia 1993)

Функция Грина разбивается на сингулярную и гладкую части:

$$G_k(R) = \underbrace{\frac{1}{4\pi R}}_{G_0 \text{ (сингулярная)}} + \underbrace{\frac{e^{ikR} - 1}{4\pi R}}_{G_{\text{smooth}}} \tag{15}$$

Гладкая часть конечна при $R = 0$:

$$G_{\text{smooth}}(0) = \lim_{R \to 0} \frac{e^{ikR} - 1}{4\pi R} = \frac{ik}{4\pi} \tag{16}$$

и интегрируется стандартной квадратурой.

7.3 Аналитический интеграл (Graglia)

Сингулярная часть G_0 обрабатывается аналитически. Ключевой интеграл — потенциал плоского треугольника:

$$P(\mathbf{r}) = \int_T \frac{dS'}{|\mathbf{r} - \mathbf{r}'|} \tag{17}$$

Формула Грайлии выражает $P(\mathbf{r})$ через сумму по трём рёбрам треугольника:

$$P(\mathbf{r}) = \sum_{i=1}^{3} \left[d_i \ln \frac{R_i^+ + s_i^+}{R_i^- + s_i^-} - |h| \left(\arctan \frac{d_i s_i^+}{h^2 + |d_i| R_i^+} - \arctan \frac{d_i s_i^-}{h^2 + |d_i| R_i^-} \right) \right] \tag{18}$$

Обозначения для i-го ребра (от вершины $\mathbf{v}_i$ к $\mathbf{v}_{i+1}$):

- $\hat{\mathbf{t}}_i$ — единичный вектор вдоль ребра
- $\hat{\mathbf{n}}$ — нормаль к треугольнику
- $\hat{\mathbf{m}}_i = \hat{\mathbf{n}} \times \hat{\mathbf{t}}_i$ — внутренняя нормаль к ребру в плоскости треугольника
- $d_i = (\mathbf{r} - \mathbf{v}_i) \cdot \hat{\mathbf{m}}_i$ — расстояние от проекции $\mathbf{r}$ до линии ребра
- $h = (\mathbf{r} - \mathbf{v}_i) \cdot \hat{\mathbf{n}}$ — высота над плоскостью треугольника
- $s_i^\pm = (\mathbf{v}_{i+1,i} - \mathbf{r}) \cdot \hat{\mathbf{t}}_i$ — проекции вершин ребра
- $R_i^\pm = |\mathbf{v}_{i+1,i} - \mathbf{r}|$ — расстояния до вершин ребра

7.4 Какие интегралы обрабатываются аналитически

В текущей реализации аналитически вычисляется только **скалярная (div-div)** часть сингулярного вклада, т.к. дивергенции $\nabla \cdot \mathbf{f}$ — константы на треугольнике:

$$L_{mn}^{\text{sing,scalar}} = -\frac{i}{k} (\nabla \cdot \mathbf{f}_m)(\nabla \cdot \mathbf{f}_n) \cdot \frac{1}{4\pi} \int_{T_{\text{test}}} P(\mathbf{r})\, dS \tag{19}$$

Внешний интеграл $\int P(\mathbf{r}) dS$ по тестовому треугольнику вычисляется квадратурой: значения $P(\mathbf{r}_q)$ в квадратурных точках суммируются с весами.

Векторная часть ($\mathbf{f}_m \cdot \mathbf{f}_n \cdot G_0$) — слабо сингулярна (умножение на ограниченные $\mathbf{f}$ улучшает сходимость) и покрывается квадратурой G_{smooth}, которая конечна при $R = 0$.

7.5 Оператор K: сингулярная часть

Для $\mathcal{K}$ при совпадающих треугольниках возникает интеграл главного значения (PV). Для **плоских копланарных** треугольников (что справедливо при $T_m = T_n$) этот PV-интеграл равен нулю по симметрии. Поэтому в коде сингулярные пары для K просто пропускаются:

$$\text{PV} \int_T \nabla G_0 \times \mathbf{f}_n \, dS' = 0 \quad (\text{при плоском } T) \tag{20}$$

7.6 Градиент потенциала (конечные разности)

Функция `gradient_potential_integral_triangle` вычисляет $\nabla_{\mathbf{r}} P(\mathbf{r})$ центральными конечными разностями с шагом $\varepsilon = 10^{-6}$:

$$\frac{\partial P}{\partial x_d} \approx \frac{P(\mathbf{r} + \varepsilon \hat{\mathbf{e}}_d) - P(\mathbf{r} - \varepsilon \hat{\mathbf{e}}_d)}{2\varepsilon} \tag{21}$$

Эта функция определена, но не используется в основной сборке (оставлена как утилита).

8 Сборка PMCHWT системы

Функция `assemble_pmchwt` собирает блочную матрицу $2N \times 2N$:

$$Z = \begin{pmatrix} \eta_1 L_1 + \eta_2 L_2 & K_1 + K_2 \\ -(K_1 + K_2) & L_1/\eta_1 + L_2/\eta_2 \end{pmatrix} \tag{22}$$

Для этого дважды вызывается `assemble_L_K`: для внешнего (k_1) и внутреннего (k_2) волновых чисел. Обе сборки выполняются на **одном и том же** меше поверхности — разница только в значении k.

9 Правая часть: падающая волна

Функция `compute_rhs_planewave` вычисляет вектор правой части $\mathbf{b}$, проецируя тангенциальные следы падающего поля на RWG-функции:

$$b_n^{(J)} = \int_S \mathbf{f}_n(\mathbf{r}) \cdot \left[\hat{\mathbf{n}} \times \mathbf{E}^{\text{inc}}(\mathbf{r})\right] dS \approx \int_S \mathbf{f}_n \cdot \mathbf{E}^{\text{inc}}\, dS \tag{23}$$

$$b_n^{(M)} = \int_S \mathbf{f}_n(\mathbf{r}) \cdot \left[\hat{\mathbf{n}} \times \mathbf{H}^{\text{inc}}(\mathbf{r})\right] dS \approx \int_S \mathbf{f}_n \cdot \mathbf{H}^{\text{inc}}\, dS \tag{24}$$

Здесь используется свойство RWG-функций: они лежат в тангенциальной плоскости треугольника, поэтому проекция $\hat{\mathbf{n}} \times \mathbf{E}$ на $\mathbf{f}_n$ упрощается (в коде тестирование производится напрямую скалярным произведением $\mathbf{f}_n \cdot \mathbf{E}_0$, что корректно для RWG).

Интегралы вычисляются квадратурой:

$$b_n = \sum_{\pm} \sum_q \mathbf{f}_n(\mathbf{r}_q) \cdot \mathbf{E}_0\, e^{ik\hat{\mathbf{k}}\cdot\mathbf{r}_q}\, w_q A \tag{25}$$

10 Решение системы и дальнее поле

10.1 Решение линейной системы

Система $Z\mathbf{c} = \mathbf{b}$ решается прямым методом (`np.linalg.solve` — LU-разложение). Вектор решения $\mathbf{c}$ содержит коэффициенты:

$$\mathbf{c} = \begin{pmatrix} \mathbf{c}_J \\ \mathbf{c}_M \end{pmatrix}, \qquad \mathbf{J}(\mathbf{r}) = \sum_{n=1}^{N} c_J^{(n)} \mathbf{f}_n(\mathbf{r}), \quad \mathbf{M}(\mathbf{r}) = \sum_{n=1}^{N} c_M^{(n)} \mathbf{f}_n(\mathbf{r}) \tag{26}$$

10.2 Вычисление дальнего поля

В дальней зоне ($r \to \infty$) рассеянное поле записывается как:

$$\mathbf{E}^{\text{sca}}(\mathbf{r}) \approx \frac{e^{ikr}}{r}\mathbf{F}(\hat{\mathbf{r}}) \tag{27}$$

Амплитуда рассеяния $\mathbf{F}$ вычисляется через интегралы токов:

$$\mathbf{F}(\hat{\mathbf{r}}) = -\frac{ik}{4\pi}\left[\mathbf{J}_\perp(\hat{\mathbf{r}}) + \hat{\mathbf{r}} \times \mathbf{M}_t(\hat{\mathbf{r}})\right] \tag{28}$$

где

$$\mathbf{J}_t(\hat{\mathbf{r}}) = \int_S \mathbf{J}(\mathbf{r}')e^{-ik\hat{\mathbf{r}}\cdot\mathbf{r}'}dS', \qquad \mathbf{J}_\perp = \mathbf{J}_t - \hat{\mathbf{r}}(\hat{\mathbf{r}}\cdot\mathbf{J}_t) \tag{29}$$

и аналогично для $\mathbf{M}_t$.

Проекции на сферические компоненты:

$$F_\theta = \mathbf{F}\cdot\hat{\boldsymbol{\theta}}, \qquad F_\phi = \mathbf{F}\cdot\hat{\boldsymbol{\phi}} \tag{30}$$

11 Сечения рассеяния

11.1 Дифференциальное сечение

$$\frac{d\sigma}{d\Omega}(\theta,\phi) = \frac{|F_\theta|^2 + |F_\phi|^2}{|E_0|^2} \tag{31}$$

11.2 Полное сечение рассеяния

$$C_{\text{sca}} = \int \frac{d\sigma}{d\Omega}\,d\Omega = 2\pi\int_0^\pi \frac{d\sigma}{d\Omega}(\theta)\sin\theta\,d\theta \quad \text{(при азимутальной симметрии)} \tag{32}$$

В общем случае (без азимутальной симметрии) нужно интегрировать по ϕ; здесь при $\phi = 0$ интеграция — одномерная (используется симметрия сферы).

11.3 Эффективность рассеяния

$$Q_{\text{sca}} = \frac{C_{\text{sca}}}{\pi a^2} \tag{33}$$

где a — радиус сферы, πa^2 — геометрическое сечение.

11.4 Оптическая теорема

Сечение экстинкции можно найти без полного углового интегрирования:

$$C_{\text{ext}} = \frac{4\pi}{k}\,\text{Im}\left[\mathbf{F}(\hat{\mathbf{k}})\cdot\hat{\mathbf{e}}_{\text{pol}}\right] \tag{34}$$

где $\hat{\mathbf{k}}$ — направление вперёд, $\hat{\mathbf{e}}_{\text{pol}}$ — поляризация. Это используется в `test_pec.py` и в скриптах сравнения с bempp-cl.

12 Теория Ми (валидация)

12.1 Коэффициенты Ми

Для сферы радиуса a параметр размера $x = ka$, относительный показатель преломления m. Коэффициенты Ми:

$$a_n = \frac{m\psi_n(mx)\psi_n'(x) - \psi_n(x)\psi_n'(mx)}{m\psi_n(mx)\xi_n'(x) - \xi_n(x)\psi_n'(mx)} \tag{35}$$

$$b_n = \frac{\psi_n(mx)\psi_n'(x) - m\psi_n(x)\psi_n'(mx)}{\psi_n(mx)\xi_n'(x) - m\xi_n(x)\psi_n'(mx)} \tag{36}$$

где функции Риккати–Бесселя:

$$\psi_n(z) = z\, j_n(z), \qquad \xi_n(z) = z\, h_n^{(1)}(z) = z[j_n(z) + i\, y_n(z)] \tag{37}$$

j_n, y_n — сферические функции Бесселя 1-го и 2-го рода.

Число членов: $n_{\max} \approx x + 4x^{1/3} + 2$ (эмпирическое правило).

12.2 Эффективности

$$Q_{\text{ext}} = \frac{2}{x^2} \sum_{n=1}^{n_{\max}} (2n+1)\, \mathrm{Re}(a_n + b_n) \tag{38}$$

$$Q_{\text{sca}} = \frac{2}{x^2} \sum_{n=1}^{n_{\max}} (2n+1)(|a_n|^2 + |b_n|^2) \tag{39}$$

12.3 PEC-сфера

При $m \to \infty$:

$$a_n^{\text{PEC}} = \frac{\psi_n(x)}{\xi_n(x)}, \qquad b_n^{\text{PEC}} = \frac{\psi_n'(x)}{\xi_n'(x)} \tag{40}$$

13 Сравнение методов

Скрипты сравнения тестируют три подхода на задаче рассеяния на сфере:

13.1 BEM (собственная реализация и bempp-cl)

- Поверхностная дискретизация: DOFs $\sim O(x^2)$
- Плотная матрица: сборка $O(N^2)$, решение $O(N^3)$ (LU)
- GMRES: $O(N^2)$ на итерацию (плотное умножение), число итераций зависит от числа обусловленности
- С FMM: $O(N \log N)$ на итерацию — масштабируемый подход

13.2 DDA / ADDA

- Объёмная дискретизация: DOFs $\sim O(x^3)$ (число диполей)
- FFT-ускоренный итерационный решатель: $O(N \log N)$ на итерацию
- Для больших частиц: $N_{\text{DDA}} \gg N_{\text{BEM}}$

13.3 Ключевые выводы из сравнений

1. Оба метода дают $\sim$1% точность на тестовых случаях ($x = 1$–5).
2. BEM выигрывает по DOFs для больших x (поверхность vs объём).
3. ADDA (DDA) проще в реализации для произвольных форм (не нужен меш).
4. Для больших задач ($x \gg 10$) BEM+FMM — наиболее эффективный подход.

14 Солверы и предобуславливание

14.1 LU-разложение (прямой метод)

Используется по умолчанию в `bem_core.py`. Сложность $O(N^3)$, но гарантирует решение. Подходит для $N \lesssim 5000$.

14.2 GMRES

В `fast_bempp.py` тестируются варианты GMRES:

- **Чистый GMRES**: restart=500, без предобуславливателя
- **GMRES + диагональное масштабирование**: $M = \text{diag}(Z)^{-1}$ — простой предобуславливатель

Вывод из экспериментов: PMCHWT-формулировка уже достаточно хорошо обусловлена для умеренных x, и GMRES сходится за несколько десятков итераций.

14.3 FMM-ускорение

В `compare_fmm.py` и `fmm_test*.py` используется FMM-ассемблер bempp-cl (через ExaFMM). Это позволяет избежать сборки плотной матрицы и выполнять матрично-векторное умножение за $O(N \log N)$. При $N > 5000$ FMM начинает выигрывать у плотного подхода.

15 Как запускать

15.1 Базовый тест (собственный BEM)

```
cd ~/bem_solver
python test_sphere.py    #

python test_pec.py       # PEC
```

Параметры в `test_sphere.py`:

- `wavelength = 2*pi` $\Rightarrow k = 1$
- `radius = 1.0` $\Rightarrow x = ka = 1$
- `m_rel = 1.5` — показатель преломления
- `refine = 1` — уровень измельчения (120 RWG, быстро)

Для более точного результата увеличьте `refine` до 2 или 3 (но время растёт как $\sim 4^{2r}$).

15.2 Сравнение с bempp-cl и ADDA

```
python compare_final.py   # BEM vs DDA vs Mie
python compare_fmm.py     # Dense vs FMM vs ADDA
```

Требуют установленных `bempp-cl` и ADDA.

15.3 Типичный вывод

```
=== BEM Solver Test: Dielectric Sphere ===
  x = k*a = 1.00
  m = 1.5
  Mie reference: Q_ext = 1.198538, Q_sca = 0.856855
  Mesh: 42 vertices, 80 triangles
  RWG basis functions: 120
  System size: 240 x 240
  Assembly time: 3.2 s
  Q_sca (BEM):  0.823456
  Q_sca (Mie):  0.856855
  Relative error: 3.9%
```

При `refine=2` (480 RWG) ошибка падает до $\sim$1%.

16 Ограничения и возможные улучшения

1. **Только замкнутые поверхности**: меш должен быть замкнутым (нет открытых рёбер).
2. **Плотная матрица**: сложность $O(N^2)$ по памяти и $O(N^3)$ по времени (LU). Для $N > 5000$ нужна FMM/H-матрица.
3. **Одна частица**: код не поддерживает множественные тела.

4. **Сингулярность вектора**: аналитически обрабатывается только скалярная (div-div) часть; векторная часть ($\mathbf{f} \cdot \mathbf{f}/R$) покрывается квадратурой G_{smooth}, что вносит небольшую погрешность.

5. **Нет адаптивной квадратуры**: для близких (но не совпадающих) треугольников квадратура не уточняется.

6. **Только сферический меш**: генератор умеет только икосферу. Для произвольных форм нужен внешний мешер.

17 Список литературы

1. S.M. Rao, D.R. Wilton, A.W. Glisson, “Electromagnetic scattering by surfaces of arbitrary shape”, IEEE TAP, 1982.

2. R.D. Graglia, “On the numerical integration of the linear shape functions times the 3-D Green’s function or its gradient on a plane triangle”, IEEE TAP, 1993.

3. A.J. Poggio, E.K. Miller, “Integral equation solutions of three-dimensional scattering problems”, in Computer Techniques for Electromagnetics, Pergamon, 1973.

4. W.C. Chew, J.-M. Jin, E. Michielssen, J. Song, “Fast and Efficient Algorithms in Computational Electromagnetics”, Artech House, 2001.

5. C.F. Bohren, D.R. Huffman, “Absorption and Scattering of Light by Small Particles”, Wiley, 1983.

6. M.A. Yurkin, A.G. Hoekstra, “The discrete-dipole-approximation code ADDA: capabilities and known limitations”, JQSRT, 2011.